The Topological Trouble With Transformers

Michael C. Mozer, Shoaib Ahmed Siddiqui, Rosanne Liu

ABSTRACT

Transformers encode structure in sequences via an expanding contextual history. However, their purely feedforward architecture fundamentally limits dynamic state tracking. State tracking – the iterative updating of latent variables reflecting an evolving environment – involves inherently sequential dependencies that feedforward networks struggle to maintain. Consequently, feedforward models push evolving state representations deeper into their layer stack with each new input step, rendering information inaccessible in shallow layers and ultimately exhausting the model's depth. While this depth limit can be bypassed by dynamic depth models and by explicit or latent thinking that externalizes state representations, these solutions are computationally and memory inefficient. In this article, we argue that temporally extended cognition requires refocusing from explicit thought traces to implicit activation dynamics via recurrent architectures. We introduce a taxonomy of recurrent and continuous-thought transformer architectures, categorizing them by their recurrence axis (depth versus step) and their ratio of input tokens to recurrence steps. Finally, we outline promising research directions, including enhanced state-space models and coarse-grained recurrence, to better integrate state tracking into modern foundation models.

Automated review. See the accompanying `peer-review-llmxive.pdf` for the full reviewer report.

llmXive follow-up. A separate llmXive study building on this work: `PROJ-931-llmxive-follow-up-extending-the-topologi`.

The Topological Trouble With Transformers

Michael C. Mozer
Google DeepMind
mcmozer@google.com

Shoaib Ahmed Siddiqui
Google DeepMind
shoaibasidd@google.com

Rosanne Liu
Google DeepMind
rosanneliu@google.com

Abstract

Transformers encode structure in sequences via an expanding contextual history. However, their purely feedforward architecture fundamentally limits dynamic state tracking. State tracking—the iterative updating of latent variables reflecting an evolving environment—involves inherently sequential dependencies that feedforward networks struggle to maintain. Consequently, feedforward models push evolving state representations deeper into their layer stack with each new input step, rendering information inaccessible in shallow layers and ultimately exhausting the model's depth. While this depth limit can be bypassed by dynamic depth models and by explicit or latent thinking that externalizes state representations, these solutions are computationally and memory inefficient. In this article, we argue that temporally extended cognition requires refocusing from explicit thought traces to implicit activation dynamics via recurrent architectures. We introduce a taxonomy of recurrent and continuous-thought transformer architectures, categorizing them by their recurrence axis (depth versus step) and their ratio of input tokens to recurrence steps. Finally, we outline promising research directions, including enhanced state-space models and coarse-grained recurrence, to better integrate state tracking into modern foundation models.

1 Introduction

Progress in understanding human cognition has resulted from conceptualizing the brain as a dynamical system. In terms of its hardware, the physical brain is composed of billions of interacting neurons whose collective behavior is inherently dynamical. In terms of its function, the emergent mind can be usefully modeled as a dynamical process with a high-dimensional state, s, that evolves over time, modulated by external stimuli, x. These levels can be bridged by formalizing the state progression as $s_t = f(s_{t-1}, x_t)$, assuming discrete time t.

From this perspective, an ideal architecture for modeling temporally extended cognition would be a recurrent neural network (RNN), which explicitly performs such a state-update operation. In principle, gradient-based training procedures might discover the function f from data such that the important input signals would be integrated into the state representation and held until later required. The appeal of RNNs was somewhat dampened in the 1990s by the inherent limitations of gradient-based training (Mozer, 1992; Hochreiter, 1998; Hochreiter et al., 2001).

Until the transformer (Vaswani et al., 2017) came along, feedforward nets did not seem like a viable approach to replicating human thought and reasoning. The transformer, with an audaciously long context window, retains all information in its history, often postponing the selection of relevant data until required for inference (Meng et al., 2022). In contrast, RNNs filter information as it arrives into a bottlenecked state representation (Hochreiter and Schmidhuber, 1997). This article is about what can go wrong with the transformer's strategy and approaches that can address its limitations.

Preprint.

2 State tracking

Tracking the evolving world state is an essential ingredient for language understanding and reasoning, regardless of how tracking is achieved. The transformer's strategy often leverages its capacity to retrieve static, previously observed information from a context window. The attention mechanism of transformers is highly effective at retrieving past tokens (Olsson et al., 2022). However, this lookup mechanism is conceptually distinct from the explicit maintenance of a dynamic state—the iterative, inherently sequential updating of latent variables that reflect a changing environment. The term *belief state* is often used to refer to this compact, sufficient summary of the knowledge an AI agent has about its environment (Chrisman, 1992; Kaelbling et al., 1998). Belief state can be a set of facts or it can be a probability distribution over possible worlds.

To illustrate, consider the game of twenty questions. Each answer to a question narrows the hypothesis space, and each subsequent question should be designed to shrink the hypothesis space further. In a game where one is asked to guess a number, if one is told that the number is larger than 50, it would make no sense to follow up it with a guess of 25. And on the flip side, if one is asked to think of a number between 1 and 100 and respond 'higher' or 'lower' to guesses, maintaining the state—the hidden number—is critical to preventing inconsistent answers. Yet, here is a trace from Gemini 3 (Fast) revealing a failure mode of models:[1]

```
User:  Think of a number between 1 and 100 and I'll guess.  Please respond
only with "higher" or "lower" or "you got it".  My first guess is 60.
Model:  lower
User:  41
Model:  lower
User:  70
Model:  higher
```

Even though the model cannot generate a random number internally, it can play the game without actually having a number in mind simply by choosing responses consistent with its previous responses. However, consistency requires tracking the valid range. Gemini 3 Thinking *does* generate an explicit target, but then it fails to make use of the generated target even though it is also a part of the input token stream in this case:

```
User:  Think of a number between 1 and 100 and I'll guess.  Please respond
only with "higher" or "lower" or "you got it".  My first guess is 60.
Model (thinking):  I've begun designing the core game mechanics.  I've
successfully established the need for a number within the range of 1 to
100.  The user's initial input, which is 60, is noted.  Now I've internally
selected the hidden number, choosing 42.  Since 60 is higher than 42, my next
step is to correctly communicate the response "lower".
Model:  lower
User:  42
Model:  lower
```

Beyond games, state tracking is essential to understanding the ever-changing world, the structure of arguments, and social interactions. State tracking failures in foundation models lead to loss of coherence in multi-turn conversations (Laban et al., 2025), inefficiency in information gathering (Sawyer et al., 2025), and breakdowns in communication and cooperation in multi-agent settings (Davidson et al., 2025; Khatua et al., 2026).

Without proper state tracking, models flip-flop in their interpretations and fail to detect their inconsistencies, e.g., the meaning of a polysemous word (Lepori et al., 2025):[2]

[1]This example was originally suggested by Gamal ElSayed circa 2020, still causing some modern foundation models to fail. Other models may have been trained to address this particular failure mode. Baldelli et al. (2026) formally address the failure of standard models to reliably maintain a consistent hidden state over sequential interactions.

[2]The example that follows was produced by Gemini 2.5 Flash in 2025. Although the model sometimes responded correctly, and newer and more powerful models are much less susceptible to this error, the example reveals a fundamental limitation of the core architecture.

```
User:  Fred took the day off work and pulled out his fishing pole.  He drove
to the bank.  When he reaches the bank, should he wear boots or sandals?
Model:  While sandals may be more comfortable for a casual day, they are not
the best choice for fishing, especially at a river bank.  Here's why:  ...
User:  Is it likely that Fred will find an ATM at this bank?
Model:  It is highly likely that Fred will find an ATM at the bank.  Most
banks, especially those located near bodies of water popular for fishing,
have ATMs on-site for customer convenience.
```

In this example, the model jumps from one interpretation of bank to the other without a human-like acknowledgment of the reversal (e.g., "Oh wait, I misinterpreted. You must have been talking about a financial institution."). We consider this flip-flop a failure to track the world situation, the model's earlier responses, and the listener's expectations.

While tracking Fred's location in the above example should not be challenging, in the most general case, it is untenable for models—and people—to maintain and track probabilistic belief states over all environmental possibilities because the distributions explode in dimensionality. People adopt heuristics such as sampling (e.g., Vul et al., 2014), collapsing distributions into prototypical cases (Tversky and Kahneman, 1971), or forming concrete mental models most consistent with premises (Johnson-Laird, 1983), like a MAP estimate. Nonetheless, even finite-memory, deterministic state tracking can be unreliable in a transformer decoder.

We will use a schematic to explain the challenge of state tracking. In Figure 1a, we depict the transformer with input steps shown along the horizontal axis and blocks (or layers) of the transformer along the vertical axis. Activation propagates from bottom to top (shallow to deep layers). For three selected blocks, we use color to indicate the functional connectivity of a causal transformer: activation in a block is influenced by all blocks immediately below and below to the left. In Figure 1b, we depict the flow of state information, indicated by the green rectangles. The integration of the state representation and a new input (purple arrows) leads to a new state representation, $s_t = f(s_{t-1}, x_t)$. Because the architecture is feedforward, s_t must lie deeper in the stack than s_{t-1}, eventually topping out of the model. Figure 2 shows examples from several recent articles matching this upward activation flow. This flow can make the integration of information over the sequence unreliable (Biran et al., 2024; Grant et al., 2025; Lepori et al., 2025; Sawyer et al., 2025; Venhoff et al., 2025; Baldelli et al., 2026).

Not every state-tracking problem requires depth linear in the number of layers. Merrill and Sabharwal (2025) prove the necessity and sufficiency of $\log n$ layers to recognize regular language strings of up to length n and graph-connectivity problems with n vertices. However, this proof addresses only the constructability of solutions, not their learnability.

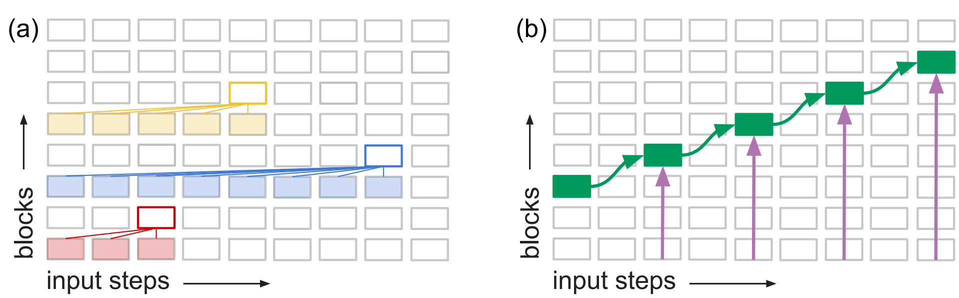

Figure 1: (a) Schematic depiction of transformer decoder architecture with input steps along horizontal axis and blocks (layers) along vertical axis. Activation flows upwards, from shallow to deep layers. For three selected blocks outlined in color, the functional input connectivity of a causal model is shown by colored lines and shading of the input sources. (b) The propagation of state representations, indicated by the green rectangles. In cases where these representations depend on previous states and current inputs, the state tracking capability of a feedforward transformer is fundamentally limited by the depth of the model.

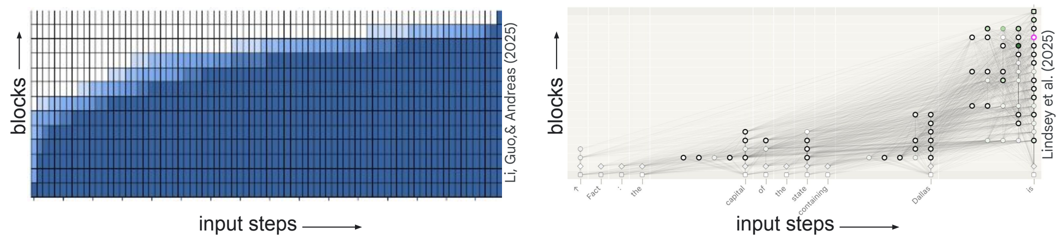

Figure 2: Two examples from the literature: Li et al. (2025a) and Lindsey et al. (2025). The details of each Figure are not critical, but in each case, the upward flow of information to deeper layers is depicted.

In practice, many researchers have identified clever solutions obtained by training depth-limited models on specific finite sequence-length problems (Li et al., 2025a; Piotrowski et al., 2025; Prakash et al., 2026; Shai et al., 2024). Essentially, when the state update function, f is of a certain form, the sequence of state updates may be composed into a simpler one-step function, e.g., there exists a function g computable by a transformer layer such that

$$s_t = f(\dots f(f(s_0, x_1), x_2), \dots, x_t) = g(s_0, x_1, \dots, x_t).$$

Training losses have been proposed that aim to steer models toward such solutions, to the extent they exist exactly or approximately (Hu et al., 2025; Teoh et al., 2025a; Huang et al., 2026). However, when belief-state cascades push deeper and deeper into a network, computational limitations arise because the resulting representations are unavailable to shallower layers. To illustrate, Figure 3 depicts the processing of the bank dialog. Using a technique called Patchscopes (Ghandeharioun et al., 2024), Lepori et al. (2025) observed that the embedding of the polysemous word 'bank' was ambiguous (i.e., a mixture of money bank vs. river bank) at shallow layers of the network, but deep in the network, the model selected the river bank interpretation. The previous sentences provided context (e.g., 'took the day off work', 'fishing pole') to support one interpretation over the other. In the Figure, we depict this contextualization as occurring at the sixth block of the transformer stack, which means that when subsequent tokens are processed, the disambiguation is not available in blocks 1-5. Lepori et al. (2025) show that this delayed disambiguation leads to downstream errors whenever the response generation outpaces the model's internal semantic convergence, such as when the model forms its yes/no response to the question about an ATM at the bank.

If reasonable models make errors when the inference cascade is just two steps (Lepori et al. studied Gemma2-9B), it should be no surprise that models produce more severe failures in comprehending extended multi-agent conversations.

One solution to the depth dilemma is chain-of-thought style "thinking" where the model recasts a deep representation as one or more output tokens, which are then available to the model on its input (Wei et al., 2022). For ordinary, step-by-step microcognition, this solution is a cop out. Inferences that people make automatically and unconsciously and then utilize consistently, such as the selection of a polysemous word's meaning, should not require elaborated, extended cognition. Even those who disagree with this desideratum should agree that if cognition in a transformer can be shifted from explicit thought traces to implicit activation dynamics, the resulting model will be more powerful.

3 Recurrent architectures

As we previously mentioned, the state tracking ability of a feedforward model is limited by model depth (e.g., Merrill and Sabharwal, 2023; Strobl et al., 2024; Merrill and Sabharwal, 2025) and by the fact that effectively utilizing the state representation becomes more challenging as it shifts upwards to deeper layers (Biran et al., 2024; Lepori et al., 2025; Sawyer et al., 2025; Venhoff et al., 2025).

The alternative is a recurrent model, which is necessary to express arbitrary state dynamics, i.e., $s_t = f(s_{t-1}, x_t)$. In this section, we explore how to combine recurrence with transformers. Given the subtle yet critical differences among varieties of recurrence, they are easily conflated and mistakenly

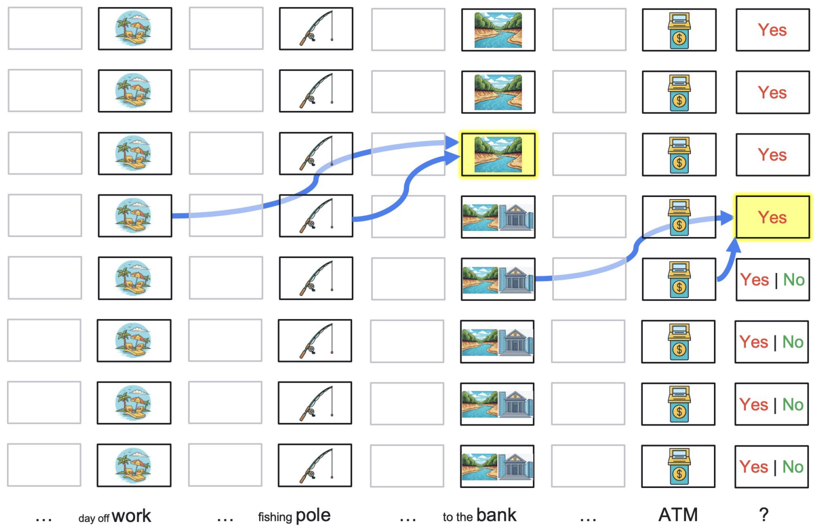

Figure 3: The depth of a state representation in a transformer can limit its utility for inference (adapted from Lepori et al., 2025). The input tokens are presented at the bottom, processed sequentially by the layers of the model. The symbols within the layers represent the model's internal belief state. This highlights that despite the model converging to the correct belief (river bank) deep in the network at the "bank" token, this high-level disambiguation is inaccessible to the shallower layers of subsequent tokens. Consequently, when processing the final tokens, the model defaults to superficial associations (money bank → ATM) early in the stack, resulting in an incorrect prediction at the last token position.

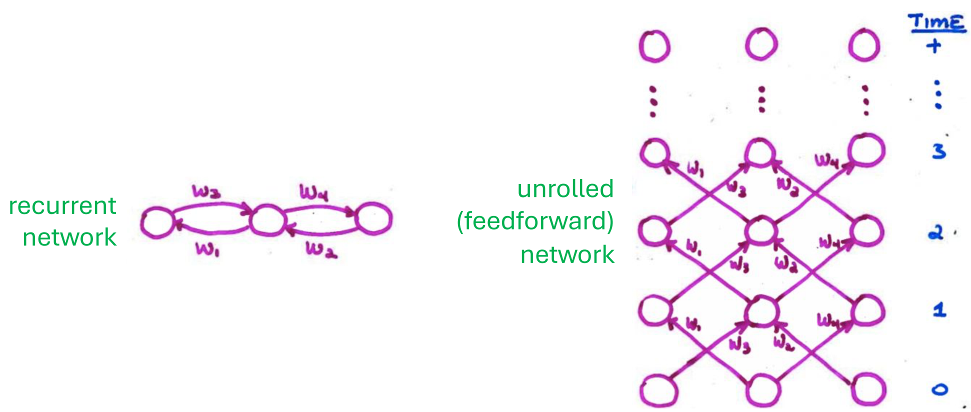

Figure 4: Unrolling a three-neuron recurrent neural net (left) into its equivalent feedforward network (right). Figure adapted from Rumelhart et al. (1986).

treated as equivalent. This equivalence is problematic because recurrence is not in itself sufficient for state tracking; the most popular form of recurrence is unable to track state (Merrill et al., 2025).

Figure 4, adapted from Rumelhart et al. (1986), shows a simple recurrent net (left) and the net unrolled t steps to form a weight-constrained feedforward net (right). Unrolling the net requires making a copy of all neurons in the network for each step of recurrence. At each step, activation flows through each connection.

Now consider a transformer with some recurrent connections, as depicted in Figure 5a. As before, each box denotes a transformer block and the horizontal and vertical axes correspond to input tokens and layers, respectively. The additional arrows are meant to illustrate one possible type of recurrence involving activation flow from a deep layer to a shallow layer at every input step. The nature of the activation flow is not important for our purposes; for example, the shallow layer may generate queries that allow it to cross attend to keys and values from the deep layer (e.g., Fan et al., 2021).

Unlike the simple RNN in Figure 4, which can be unrolled in only one way, unrolling the recurrent transformer in Figure 5a results in ambiguity due to the fact that that the transformer architecture incorporates three distinct ordered dimensions: (1) the layers of the architecture, bottom to top (shallow to deep) in the Figure; (2) the input steps of the architecture, left to right in the Figure; and (3) autoregressive steps performed at execution. During model pretraining, an ordinary (feedforward) transformer has only a single autoregressive step because all input steps are run in parallel; and during inference, autoregressive steps are typically confounded with input steps, although multi-token prediction allows for multiple input/output steps per autoregressive step (Gloeckle et al., 2024). However, recurrence allows for further decoupling. In particular, many forms of recurrence require that even when a model is trained via teacher forcing, it must still be unrolled autoregressively (Teoh et al., 2025b). This necessary sequentiality is what we mean by *autoregressive unrolling*, not the sequentiality that arises from token-by-token generation in a pure feedforward model.

Figure 5b depicts a transformer unrolled in depth, sometimes referred to as a *looped transformer*. Depth recurrence—whether of individual layers or ranges of layers, and whether deterministic or adaptive—is a very popular and successful approach. Some methods are designed and trained to allow for inference time scaling (e.g., Yang et al., 2024a; Nowak et al., 2024; Raposo et al., 2024; Alabdulmohsin and Zhai, 2025; Bae et al., 2025; Chen et al., 2025b; Geiping et al., 2025; Rodkin et al., 2025; Yu et al., 2025; Zhu et al., 2025; Zeng et al., 2026; Jeddi et al., 2026); others incorporate recurrence via pretraining (Sanyal, 2026) or fine tuning a pretrained model (Koishekenov et al., 2025; McLeish et al., 2025); and surprisingly, several operate purely as an inference-time method to improve reasoning (Li et al., 2025b; Chen et al., 2026; Ng, 2026).

While depth recurrence can increase the expressivity of a transformer (Saunshi et al., 2025), it does not enable indefinite state tracking; the propagation is still depth limited. To appreciate this fact, pick any layer l in the lower or upper stack as the representation of $s(0)$, the state at the first stack. Then note that any $s(t+1)$, if it is to depend arbitrarily on $s(t)$ recursively, must be in a higher layer. The

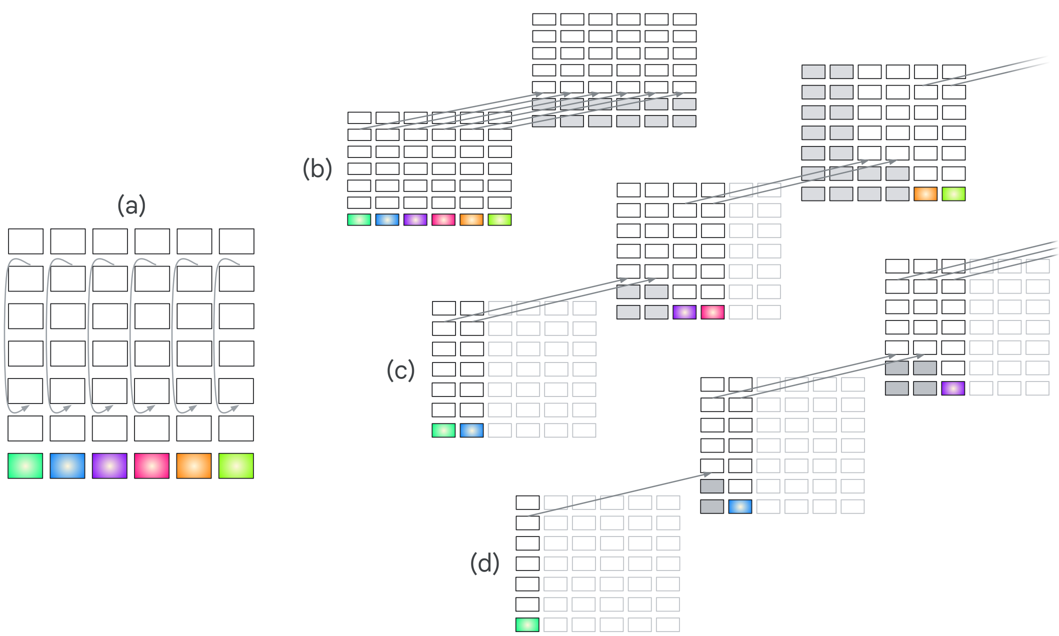

Figure 5: Unrolling a transformer. Each rectangle represents a transformer layer. The colored boxes represent distinct input tokens. The arrows indicate flow of activity. Note that the figure represents the output of the full transformer layer, which includes both the self-attention as well as the feedforward layer. The gray filled rectangles indicate blocks that can be replaced by the frozen KV cache as their values no longer change. The light grey unfilled rectangles represent input steps that are ignored at a given unrolling step. (a) A transformer with recurrence that allows activation from a deep layer to influence a shallow layer. (b) An unrolled depth-recurrent transformer. (c) An unrolled blockwise-recurrent transformer. (d) A transformer with potential attractor dynamics.

state representation still shifts upward due to the parallel propagation of activation across steps t, regardless of how deep the transformer is made to be with recurrent depth (Figure 1b).

Indefinite tracking of state with an arbitrary state update function requires sequential dependency that precludes parallelization across the sequence length *during training*. Two examples of autoregressive updates are presented in Figures 5c,d. Figure 5c depicts a blockwise-recurrent model (Hutchins et al., 2022; Chevalier et al., 2023; Chen et al., 2025a; Borazjanizadeh and McClelland, 2025) in which a subsequence of input steps is run in parallel (two in the Figure) followed by an autoregressive iteration; Figure 5d shows a model in which one input step is presented per autoregressive step, and at step t, all stacks up to $t-1$ send a signal from the deep layer to the shallow layer, yielding a fully recurrent model. This model may have attractor dynamics since each layer continues to update, converging only when all previous steps have reached asymptote.

Having addressed recurrence with multiple input steps per autoregressive step (Figure 5c) and a standard one-to-one mapping (Figure 5d), we can also consider a setup where multiple autoregressive steps are executed for each input step (Figure 6). Latent-thought models have this form (e.g., Hao et al., 2025; Jolicoeur-Martineau, 2025). Some of these models can track state, whereas others do not (e.g., Galashov et al., 2025).

The examples of recurrence we have given all involve signal propagation from deeper layers to shallower layers, but within-layer propagation over inputs steps (Figure 7) also yields state dynamics (e.g., Allen-Zhu, 2025; Gu and Dao, 2024).

Table 1 attempts to lay out a taxonomy of recurrent transformer architectures that includes the cases we have considered so far. We characterize architectures along two dimensions: *recurrence axis* and *input tokens per recurrence step*. The axis can be depth alone (Figures 5b,c,d), in step alone (Figure 7), and in both depth and step (Figure 6). The ratio of input tokens to recurrence steps can be greater than one (Figure 5b,c), equal to one (Figure 5d, 7), or less than one (Figure 6).[3] In cells of the taxonomy, we list some popular and representative transformer-based architectures. We omit architectures that are described as "recurrent" in the sense that processing is iterative, but that are

[3]We define a 'recurrence step' strictly as a sequential dependency that precludes parallelization across the sequence length during training.

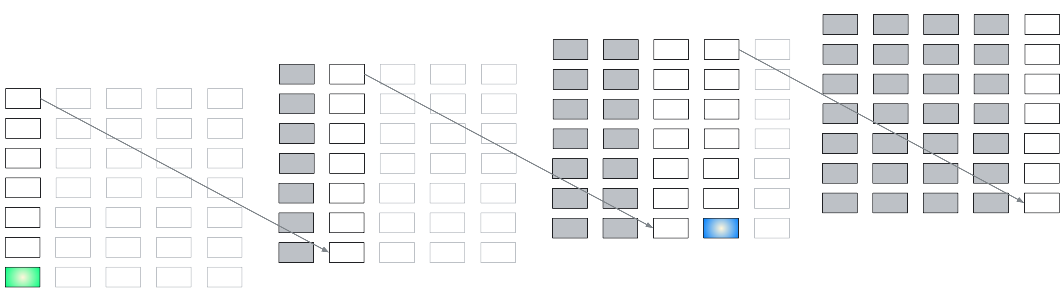

Figure 6: Unrolling a latent-thought model, where the model feeds back its latent thoughts as input to the model for multiple auto-regressive steps before processing the next input token, which is marked with the blue color (e.g., Hao et al., 2025; Jolicoeur-Martineau, 2025).

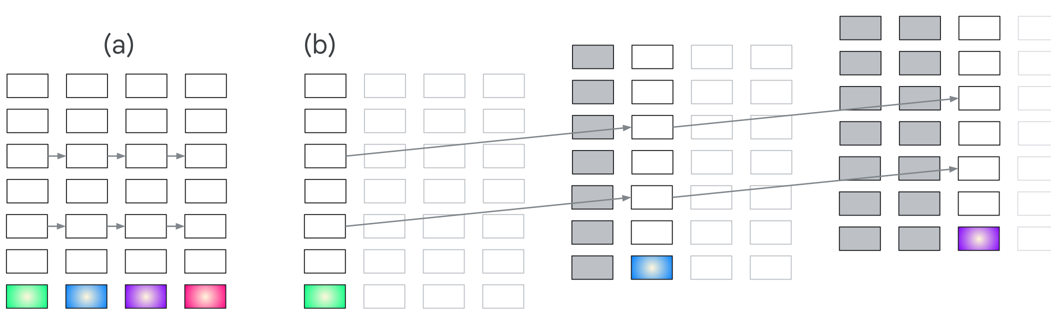

Figure 7: Unrolling an SSM, where information from the previous input step at layer l flows directly to the next step in the horizontal direction (e.g., Allen-Zhu, 2025; Gu and Dao, 2024).

Table 1: A taxonomy of recurrent transformer architectures, with selected examples. Note that ratio > 1 indicates processing multiple tokens per recurrence step, while ratio < 1 indicates multiple recurrence steps per token.

Recurrence Axis	**Input Tokens Per Recurrence Step**		
	Ratio > 1	*Ratio* $= 1$	*Ratio* < 1
Depth	[Figure 5b] looped transformer (Giannou et al., 2023), universal transformer (Dehghani et al., 2019), RINS (Alabdulmohsin and Zhai, 2025)	[Figure 5d]	
Step	block-recurrent transformers (Hutchins et al., 2022)	[Figure 7b] linear attention (Katharopoulos et al., 2020), DeltaNet (Schlag et al., 2021), MAMBA (Gu and Dao, 2024), canon layers (Allen-Zhu, 2025), PaTH attention (Yang et al., 2025b), RWKV-7 (Peng et al., 2025), test-time regression (Sun et al., 2025)	DeltaProduct (Siems et al., 2025)
Depth + Step	[Figure 5c] recurrent memory transformer (Bulatov et al., 2022), RINs (Jabri et al., 2023), sentence gestalt (Borazjanizadeh and McClelland, 2025)	feedback transformer (Fan et al., 2021)	[Figure 6] COCONUT (Hao et al., 2025), hierarchical reasoning model (Jolicoeur-Martineau, 2025), CYB (Galashov et al., 2025)

neither recurrent in depth nor step, e.g., Transformer-XL (Dai et al., 2019). Being recurrent in depth and/or step is necessary for state tracking but is not sufficient. Essentially, full-fledged state tracking requires sequential dynamics during training; any model that can be entirely parallelized across the context has limitations in updating state.

While we have not shown all such architectures, we have not succeeded in identifying any examples of work that lies in the empty cells of the taxonomy. Some of these cells would be worth exploration. For example, architectures in the second row, third column, could include ones with within-layer attractor dynamics, where the model's activation may iterate to convergence before advancing to the next token. Architectures in the first row, second and third columns, are interesting because they feed activation from a deep layer to a shallow layer (once for ratio $= 1$, multiple times for ratio > 1) in a manner that—unlike the looped transformer—does allow for indefinite state propagation.

We have deliberately sidestepped the question of what a recurrent arrow signifies in terms of activation dynamics. Different models make different proposals. The arrow could indicate copying key-value cache (Yang et al., 2024c), it could represent a source of keys and values for cross attention (Fan et al., 2021) or self attention (Oncescu et al., 2026), or it might represent a direct connection through a linear layer, an MLP, or an adapter such as LoRA (Hu et al., 2022). The coupling of step-to-step recurrence with attention—which gives neurons direct visibility to the entire step history—prevents credit assignment bottlenecks that arise when training traditional recurrent neural networks (Ke et al., 2018; Oncescu et al., 2026).

4 Architectural limitations and workarounds

Liu et al. (2026) point to the weakness of modern massively parallel architectures on problems that are inherently sequential, problems where combinatorics make it impractical to parallelize, such as state tracking, multihop inference, and planning. Formal analyses point to bounds on serial capacity for transformers (Merrill and Sabharwal, 2023). State-space models (SSMs) are often touted as a means of state propagation, but SSMs with linear updates are no more expressive than an ordinary transformer (Merrill et al., 2025). In contrast, chain of thought does enhance model expressivity (Li et al., 2024; Merrill and Sabharwal, 2024), as one would intuit: Allowing a model to talk to itself, whether in natural language or latent space, sends signals from deep in the transformer to shallow layers, thereby propagating state forward. It is no wonder that frontier models have become increasingly reliant on internal 'thinking'. However, the reliance on intermediate outputs to track micro-state may perform wasteful computation steps and unnecessarily consume the context window. Implicit activation dynamics—albeit recurrent—might be adequate to efficiently update mundane state information of the sort that people can process unconsciously and automatically. In human terms, it is fine to talk to yourself if you are reasoning through a calculus problem, but it is a bit weird to have to continually remind yourself of the relationship between two characters in a book you are reading.

Given their limitations in state tracking, why are transformers as successful as they are? The short answer is that by being able to reexamine their entire input history, they can often turn a state-tracking problem into a working memory problem, i.e., into retrieval from the context window. For example, consider the *latch* problem from the 1990s that required a sequence-processing model latch on to a bit of information early in a sequence and to retain it over a long time gap (Mozer, 1991; Bengio et al., 1994). This problem in large part motivated LSTM but is entirely trivial for a transformer that can re-index into its input history to retrieve the early information. Transformers learn many clever strategies including this sort of lookback (Prakash et al., 2026), associatve scans (Li et al., 2025a), more specialized algorithms for formal language understanding (Allen-Zhu and Li, 2025; Piotrowski et al., 2025), including belief updating that reflects state uncertainty (Shai et al., 2024). A simple illustration is the computation of parity from an input sequence of 0's and 1's. Instead of maintaining a parity state, the model can compute pairwise parity in the first layer of a transformer, then combine the pairs into four-bit parity in the second layer, and so forth, achieving a computation from scratch in $\log_2 n$ feedforward layers for a sequence of maximum length n. Such alternate and even shortcut solutions are common for transformers to develop in practice (Liu et al., 2022).

Another factor in the success of transformers is their support of *state compositionality*. State need not be—as we have depicted it in Figure 1—a monolithic representation. The representation of state can be split across embeddings and updated asynchronously for each component. For example, if the model needs to track the changing locations of two entities, those state variables can be updated independently for each entity.

5 Promising directions

In addition to the native mechanisms that transformers have to estimate state, several emerging research directions seem particularly promising for promoting state maintenance and updating. These directions balance the need for expressivity with computational feasibility.

5.1 Enhanced State-Space Models

While most linear SSMs do not exceed the expressivity of standard transformers, Delta Net (Schlag et al., 2021), when its eigenvalue range is extended to include negative values (Grazzi et al., 2025), maintains the highly desirable property of being trainable in parallel while simultaneously achieving greater expressivity than a standard transformer. The delta rule underlying Delta Net has inspired a range of further developments, including RWKV-7 (Peng et al., 2025) and PaTH attention (Yang et al., 2025b), that also achieve state tracking beyond the capability of ordinary transformers while demonstrating competitive language modeling at scale. Similarly, other new forms of attention are being developed with sequential dependencies that promise to be more powerful in stateful tasks (Lin et al., 2025; Leviathan et al., 2025; Beltagy et al., 2020), including the notion of gated linear attention (Yang et al., 2024b) and gated Delta Net (Yang et al., 2025a) that, when mixed with standard transformer blocks is more powerful than either, both in theory and practice (Merrill et al., 2026).

5.2 Approximating state tracking in feedforward transformers

Rather than incorporating recurrence, feedforward transformers can be steered to better approximate state tracking through specialized training objectives and structural priors (Hu et al., 2025; Teoh et al., 2025a; Huang et al., 2026). The promise is that such biases will encourage models to bolster their native lookback abilities. However, we hope that future research will take into account structured, compositional state representations.

5.3 Coarse recurrence

Introducing recurrence at a coarser granularity than individual tokens can mitigate the computational burden of token-by-token state updates. In the past, block-recurrent models have operated on this principle by compressing and passing memory forward in fixed- length chunks. A promising approach is to consider linguistic structure in identifying the chunks: Borazjanizadeh and McClelland (2025) have modeled language as a sequence of discrete 'thoughts' by chunking at the sentence level.

5.4 Leveraging representational alignment

Variable-depth models dynamically select the number of iterations of a layer or the number of times a range of layers repeats. The fact that this approach succeeds with only fine tuning—and sometimes with no training whatsoever—suggests that the model is well predisposed to communication of representations across layers, due to the alignment resulting from the residual connections. Similarly, canon layers (Allen-Zhu and Li, 2025) leverage alignment of representations from one input step to the next. We speculate that there are additional means of leveraging this alignment.

5.5 Efficient training of recurrence

Any architecture which is capable of indefinite, arbitrary state propagation requires autoregressive processing. Autoregressive pretraining is computationally inefficient and limits parallelization (Chevalier et al., 2023). One potential solution is a multi-stage training scheme in which initial pretraining relies entirely on standard, parallelizable feedforward transformer architectures. Recurrent mechanisms are then introduced only at a later training stage. To ensure training efficiency during these subsequent recurrent stages, optimization techniques such as truncated gradient methods and—for training attractor dynamics—recurrent backpropagation (Almeida, 1987; Pineda, 1987; Liao et al., 2018). Methods have also been proposed to increase *arithmetic intensity* to obtain scaling that is near linear time in context length versus quadratic for a naive implementation (Oncescu et al., 2026).

6 Conclusions

Although the transformer's feedforward design has expanded the limits of context-based retrieval, its topological structure remains fundamentally at odds with the iterative nature of state tracking. As we have argued, the current reliance on explicit natural-language-like "thought" to bypass depth limitations is an inefficient workaround for a structural deficiency. By transitioning toward implicit, recurrent activation dynamics, we can move beyond these depth-limited constraints to attain robust long-term coherence and multihop inference.

The taxonomy and research directions proposed in this article provide a roadmap for improving sequential inference dependencies without sacrificing the foundational strengths of modern models. Ultimately, bridging the gap between the transformer's parallel efficiency and the brain's inherent dynamical nature is essential. The next generation of foundation models must do more than simply re-scan the past; they must maintain a fluid, evolving representation of reality that persists across the many time scales required for temporally extended cognition.

Acknowledgments and Disclosure of Funding

Many thanks to Sunny Sanyal, Kazuki Irie, Kevin Murphy, Jay McClelland, and Chris Williams for helpful feedback on earlier drafts of the manuscript.

References

Alabdulmohsin, I. and Zhai, X. (2025). Recursive inference scaling: A winning path to scalable inference in language and multimodal systems. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Allen-Zhu, Z. (2025). Physics of language models: Part 4.1, architecture design and the magic of canon layers. arXiv:2512.17351 [cs.CL].

Allen-Zhu, Z. and Li, Y. (2025). Physics of language models: Part 1, learning hierarchical language structures. arXiv:2305.13673 [cs.CL].

Almeida, L. B. (1987). A learning rule for asynchronous perceptrons with feedback in a combinatorial environment. In *Proceedings of the IEEE First International Conference on Neural Networks*, volume 2, pages 609–618, San Diego, CA, USA. IEEE.

Bae, S., Kim, Y., Bayat, R., Kim, S., Ha, J., Schuster, T., Fisch, A., Harutyunyan, H., Ji, Z., Courville, A., and Yun, S.-Y. (2025). Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Baldelli, D., Parviz, A., Zouaq, A., and Chandar, S. (2026). Llms can't play hangman: On the necessity of a private working memory for language agents. arXiv:2601.06973 [cs.CL].

Beltagy, I., Peters, M. E., and Cohan, A. (2020). Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*.

Bengio, Y., Simard, P., and Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2):157–166.

Biran, E., Gottesman, D., Yang, S., Geva, M., and Globerson, A. (2024). Hopping too late: Exploring the limitations of large language models on multi-hop queries. In Al-Onaizan, Y., Bansal, M., and Chen, Y.-N., editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 14113–14130, Miami, Florida, USA. Association for Computational Linguistics.

Borazjanizadeh, N. and McClelland, J. (2025). Modeling language as a sequence of thoughts. arXiv:2512.25026 [cs.CL].

Bulatov, A., Kuratov, Y., and Burtsev, M. (2022). Recurrent memory transformer. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 11079–11091. Curran Associates, Inc.

Chen, L., Li, J., Liang, C., Lao, N., and Liu, Q. (2026). Training-free looped transformers. arXiv:2605.23872 [cs.CL].

Chen, Y., Hutchins, D., Jansen, A., Zhmoginov, A., Racz, D., and Andersen, J. S. (2025a). MELODI: Exploring memory compression for long contexts. In *The Thirteenth International Conference on Learning Representations*.

Chen, Y., Shang, J., Zhang, Z., Xie, Y., Sheng, J., Liu, T., Wang, S., Sun, Y., Wu, H., and Wang, H. (2025b). Inner thinking transformer: Leveraging dynamic depth scaling to foster adaptive internal thinking. arXiv:2502.13842 [cs.CL].

Chevalier, A., Wettig, A., Ajith, A., and Chen, D. (2023). Adapting language models to compress contexts. arXiv:2305.14788 [cs.CL].

Chrisman, L. (1992). Reinforcement learning with perceptual aliasing: the perceptual distinctions approach. In *Proceedings of the Tenth National Conference on Artificial Intelligence*, AAAI'92, page 183–188. AAAI Press.

Dai, Z., Yang, Z., Yang, Y., Carbonell, J., Le, Q. V., and Salakhutdinov, R. (2019). Transformer-xl: Attentive language models beyond a fixed-length context. arXiv:1901.02860 [cs.LG].

Davidson, T. R., Fourney, A., Amershi, S., West, R., Horvitz, E., and Kamar, E. (2025). The collaboration gap. arXiv:2511.02687 [cs.AI].

Dehghani, M., Gouws, S., Vinyals, O., Uszkoreit, J., and Kaiser, L. (2019). Universal transformers. In *International Conference on Learning Representations*.

Fan, A., Lavril, T., Grave, E., Joulin, A., and Sukhbaatar, S. (2021). Addressing some limitations of transformers with feedback memory. arXiv:2002.09402 [cs.CL].

Galashov, A., Jones, M., Ke, R., Cao, Y., Nagarajan, V., and Mozer, M. C. (2025). Catch your breath: Adaptive computation for self-paced sequence production. arXiv:2510.13879 [cs.CL].

Geiping, J., McLeish, S. M., Jain, N., Kirchenbauer, J., Singh, S., Bartoldson, B. R., Kailkhura, B., Bhatele, A., and Goldstein, T. (2025). Scaling up test-time compute with latent reasoning: A recurrent depth approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Ghandeharioun, A., Caciularu, A., Pearce, A., Dixon, L., and Geva, M. (2024). Patchscopes: a unifying framework for inspecting hidden representations of language models. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org.

Giannou, A., Rajput, S., Sohn, J.-y., Lee, K., Lee, J. D., and Papailiopoulos, D. (2023). Looped transformers as programmable computers. In *International Conference on Machine Learning*, pages 11398–11442. PMLR.

Gloeckle, F., Idrissi, B. Y., Rozière, B., Lopez-Paz, D., and Synnaeve, G. (2024). Better and faster large language models via multi-token prediction. *arXiv preprint arXiv:2404.19737*.

Grant, S., Goodman, N. D., and McClelland, J. L. (2025). Emergent symbol-like number variables in artificial neural networks.

Grazzi, R., Siems, J., Zela, A., Franke, J. K. H., Hutter, F., and Pontil, M. (2025). Unlocking state-tracking in linear rnns through negative eigenvalues. arXiv:2411.12537 [cs.LG].

Gu, A. and Dao, T. (2024). Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*.

Hao, S., Sukhbaatar, S., Su, D., Li, X., Hu, Z., Weston, J. E., and Tian, Y. (2025). Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*.

Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(02):107–116.

Hochreiter, S., Bengio, Y., Frasconi, P., and Schmidhuber, J. (2001). Gradient flow in recurrent nets: the difficulty of learning long-term dependencies.

Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural computation*, 9(8):1735–1780.

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al. (2022). Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3.

Hu, E. S., Ahn, K., Liu, Q., Xu, H., Tomar, M., Langford, A., Jayaraman, D., Lamb, A., and Langford, J. (2025). The belief state transformer. In *The Thirteenth International Conference on Learning Representations*.

Huang, H., LeCun, Y., and Balestriero, R. (2026). Semantic tube prediction: Beating LLM data efficiency with JEPA. arXiv:2602.22617 [cs.LG].

Hutchins, D., Schlag, I., Wu, Y., Dyer, E., and Neyshabur, B. (2022). Block-recurrent transformers. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A., editors, *Advances in Neural Information Processing Systems*, volume 35, pages 33248–33261. Curran Associates, Inc.

Jabri, A., Fleet, D., and Chen, T. (2023). Scalable adaptive computation for iterative generation. arXiv:2212.11972 [cs.LG].

Jeddi, A., Ciccone, M., and Taati, B. (2026). Loopformer: Elastic-depth looped transformers for latent reasoning via shortcut modulation. In *The Fourteenth International Conference on Learning Representations*.

Johnson-Laird, P. (1983). *Mental Models: Towards a Cognitive Science of Language, Inference, and Consciousness*. Cognitive science series. Harvard University Press.

Jolicoeur-Martineau, A. (2025). Less is more: Recursive reasoning with tiny networks. arXiv:2510.04871 [cs.LG].

Kaelbling, L. P., Littman, M. L., and Cassandra, A. R. (1998). Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101(1):99–134.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. (2020). Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR.

Ke, N. R., ALIAS PARTH GOYAL, A. G., Bilaniuk, O., Binas, J., Mozer, M. C., Pal, C., and Bengio, Y. (2018). Sparse attentive backtracking: Temporal credit assignment through reminding. In Bengio, S., Wallach, H., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc.

Khatua, A., Zhu, H., Tran, P., Prabhudesai, A., Sadrieh, F., Lieberwirth, J. K., Yu, X., Fu, Y., Ryan, M. J., Pei, J., and Yang, D. (2026). Cooperbench: Why coding agents cannot be your teammates yet.

Koishekenov, Y., Lipani, A., and Cancedda, N. (2025). Encode, think, decode: Scaling test-time reasoning with recursive latent thoughts. arXiv:2510.07358 [cs.LG].

Laban, P., Hayashi, H., Zhou, Y., and Neville, J. (2025). Llms get lost in multi-turn conversation. arXiv:2505.06120 [cs.CL].

Lepori, M. A., Mozer, M. C., and Ghandeharioun, A. (2025). Racing thoughts: Explaining contextualization errors in large language models. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3020–3036, Albuquerque, New Mexico. Association for Computational Linguistics.

Leviathan, Y., Kalman, M., and Matias, Y. (2025). Selective attention improves transformer. In *The Thirteenth International Conference on Learning Representations*.

Li, B. Z., Guo, Z. C., and Andreas, J. (2025a). (how) do language models track state? In *Forty-second International Conference on Machine Learning*.

Li, Z., Li, Y., and Zhou, T. (2025b). Skip a layer or loop it? test-time depth adaptation of pretrained llms. arXiv:2507.07996 [cs.LG].

Li, Z., Liu, H., Zhou, D., and Ma, T. (2024). Chain of thought empowers transformers to solve inherently serial problems. In *The Twelfth International Conference on Learning Representations*.

Liao, R., Xiong, Y., Fetaya, E., Zhang, L., Yoon, K., Pitkow, X., Urtasun, R., and Zemel, R. (2018). Reviving and improving recurrent back-propagation. In Dy, J. and Krause, A., editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 3082–3091. PMLR.

Lin, Z., Nikishin, E., He, X., and Courville, A. (2025). Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations*.

Lindsey, J., Gurnee, W., Ameisen, E., Chen, B., Pearce, A., Turner, N. L., Citro, C., et al. (2025). On the biology of a large language model. *Transformer Circuits Thread*.

Liu, B., Ash, J. T., Goel, S., Krishnamurthy, A., and Zhang, C. (2022). Transformers learn shortcuts to automata. *arXiv preprint arXiv:2210.10749*.

Liu, Y., Preechakul, K., Kuwaranancharoen, K., and Bai, Y. (2026). The serial scaling hypothesis.

McLeish, S., Li, A., Kirchenbauer, J., Kalra, D. S., Bartoldson, B. R., Kailkhura, B., Schwarzschild, A., Geiping, J., Goldstein, T., and Goldblum, M. (2025). Teaching pretrained language models to think deeper with retrofitted recurrence.

Meng, K., Bau, D., Andonian, A., and Belinkov, Y. (2022). Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372.

Merrill, W., Li, Y., Romero, T., Svete, A., Costello, C., Dasigi, P., Groeneveld, D., Heineman, D., Kuehl, B., Lambert, N., Li, C., Lo, K., Malik, S., Matusz, D., Minixhofer, B., Morrison, J., Soldaini, L., Timbers, F., Walsh, P., Smith, N. A., Hajishirzi, H., and Sabharwal, A. (2026). Olmo hybrid: From theory to practice and back. Technical report, Allen Institute for AI. Technical Report.

Merrill, W., Petty, J., and Sabharwal, A. (2025). The illusion of state in state-space models. arXiv:2404.08819 [cs.LG].

Merrill, W. and Sabharwal, A. (2023). The parallelism tradeoff: Limitations of log-precision transformers. *Transactions of the Association for Computational Linguistics*, 11:531–545.

Merrill, W. and Sabharwal, A. (2024). The expressive power of transformers with chain of thought. In *The Twelfth International Conference on Learning Representations*.

Merrill, W. and Sabharwal, A. (2025). A little depth goes a long way: The expressive power of log-depth transformers. arXiv:2503.03961 [cs.LG].

Mozer, M. C. (1991). Induction of multiscale temporal structure. *Advances in neural information processing systems*, 4.

Mozer, M. C. (1992). The induction of multiscale temporal structure. In Moody, J. E., Hanson, S. J., and Lippmann, R. P., editors, *Advances in Neural Information Processing Systems 4*, pages 275–282, San Mateo, CA. Morgan Kaufmann.

Ng, D. N. (2026). LLM neuroanatomy: How I topped the LLM leaderboard without changing a single weight. https://dnhkng.github.io/posts/rys/.

Nowak, A. I., Mercea, O.-B., Arnab, A., Pfeiffer, J., Dauphin, Y., and Evci, U. (2024). Towards optimal adapter placement for efficient transfer learning. arXiv:2410.15858 [cs.LG].

Olsson, C., Elhage, N., Nanda, N., Joseph, N., DasSarma, N., Henighan, T., Mann, B., Askell, A., Bai, Y., Chen, A., Conerly, T., Drain, D., Ganguli, D., Hatfield-Dodds, Z., Hernandez, D., Johnston, S., Jones, A., Kernion, J., Lovitt, L., Ndousse, K., Amodei, D., Brown, T., Clark, J., Kaplan, J., McCandlish, S., and Olah, C. (2022). In-context learning and induction heads. *Transformer Circuits Thread*. https://transformer-circuits.pub/2022/in-context-learning-and-induction-heads/index.html.

Oncescu, C.-A., Morwani, D., Jelassi, S., Meterez, A., Kwun, M., and Kakade, S. (2026). The recurrent transformer: Greater effective depth and efficient decoding. arXiv:2604.21215 [cs.LG].

Peng, B., Zhang, R., Goldstein, D., Alcaide, E., Du, X., Hou, H., Lin, J., Liu, J., Lu, J., Merrill, W., Song, G., Tan, K., Utpala, S., Wilce, N., Wind, J. S., Wu, T., Wuttke, D., and Zhou-Zheng, C. (2025). RWKV-7 ”goose” with expressive dynamic state evolution. In *Second Conference on Language Modeling*.

Pineda, F. (1987). Generalization of back-propagation to recurrent neural networks. *Physical Review Letters*, 59(19):2229–2232.

Piotrowski, M., Riechers, P. M., Filan, D., and Shai, A. S. (2025). Constrained belief updates explain geometric structures in transformer representations. arXiv:2502.01954 [cs.LG].

Prakash, N., Shapira, N., Sharma, A. S., Riedl, C., Belinkov, Y., Shaham, T. R., Bau, D., and Geiger, A. (2026). Language models use lookbacks to track beliefs. In *The Fourteenth International Conference on Learning Representations*.

Raposo, D., Ritter, S., Richards, B., Lillicrap, T., Humphreys, P. C., and Santoro, A. (2024). Mixture-of-depths: Dynamically allocating compute in transformer-based language models. arXiv:2404.02258 [cs.LG].

Rodkin, I., Orel, D., Smirnov, K., Bolatov, A., Elbouardi, B., Hassan, B., Kuratov, Y., Bulatov, A., Nakov, P., Baldwin, T., Shelmanov, A., and Burtsev, M. (2025). Beyond memorization: Extending reasoning depth with recurrence, memory and test-time compute scaling. arXiv:2508.16745 [cs.LG].

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *nature*, 323(6088):533–536.

Sanyal, S. (2026). Looped-gpt: Looping during pre-training improves generalization.

Saunshi, N., Dikkala, N., Li, Z., Kumar, S., and Reddi, S. J. (2025). Reasoning with latent thoughts: On the power of looped transformers. In *The Thirteenth International Conference on Learning Representations*.

Sawyer, D. P., Ke, N. R., Soyer, H., Engelcke, M., Reid, J., Reichert, D. P., Hudson, D. A., Lerchner, A., Rezende, D. J., Lillicrap, T. P., Mozer, M. C., and Wang, J. X. (2025). Exploring exploration with foundation agents in interactive environments. In *NeurIPS 2025 Workshop on Embodied World Models for Decision Making*.

Schlag, I., Irie, K., and Schmidhuber, J. (2021). Linear transformers are secretly fast weight programmers. *arXiv preprint arXiv:2102.11174*.

Shai, A., Riechers, P. M., Teixeira, L., Oldenziel, A. G., and Marzen, S. (2024). Transformers represent belief state geometry in their residual stream. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.

Siems, J., Carstensen, T., Zela, A., Hutter, F., Pontil, M., and Grazzi, R. (2025). Deltaproduct: Improving state-tracking in linear RNNs via householder products. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Strobl, L., Merrill, W., Weiss, G., Chiang, D., and Angluin, D. (2024). What formal languages can transformers express? a survey. *Transactions of the Association for Computational Linguistics*, 12:543–561.

Sun, Y., Li, X., Dalal, K., Xu, J., Vikram, A., Zhang, G., Dubois, Y., Chen, X., Wang, X., Koyejo, S., Hashimoto, T., and Guestrin, C. (2025). Learning to (learn at test time): RNNs with expressive hidden states. In *Forty-second International Conference on Machine Learning*.

Teoh, J., Tomar, M., Ahn, K., Hu, E. S., Sharma, P., Islam, R., Lamb, A., and Langford, J. (2025a). Next-latent prediction transformers learn compact world models.

Teoh, J., Tomar, M., Ahn, K., Hu, E. S., Sharma, P., Islam, R., Lamb, A., and Langford, J. (2025b). Next-latent prediction transformers learn compact world models. *arXiv preprint arXiv:2511.05963*.

Tversky, A. and Kahneman, D. (1971). Belief in the law of small numbers. *Psychological Bulletin*, 76(2):105–110.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. (2017). Attention is all you need. In Guyon, I., Luxburg, U. V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., and Garnett, R., editors, *Advances in Neural Information Processing Systems*, volume 30, pages 5998–6008. Curran Associates, Inc.

Venhoff, C., Khakzar, A., Joseph, S., Torr, P., and Nanda, N. (2025). Too late to recall: Explaining the two-hop problem in multimodal knowledge retrieval. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Vul, E., Goodman, N., Griffiths, T. L., and Tenenbaum, J. B. (2014). One and done? Optimal decisions from very few samples. *Cognitive Science*, 38(4):599–637.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.

Yang, L., Lee, K., Nowak, R. D., and Papailiopoulos, D. (2024a). Looped transformers are better at learning learning algorithms. In *The Twelfth International Conference on Learning Representations*.

Yang, S., Kautz, J., and Hatamizadeh, A. (2025a). Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*.

Yang, S., Shen, Y., Wen, K., Tan, S., Mishra, M., Ren, L., Panda, R., and Kim, Y. (2025b). PaTH attention: Position encoding via accumulating householder transformations. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.

Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. (2024b). Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, ICML'24. JMLR.org.

Yang, Y., Cao, Z., Chen, Q., Qin, L., Yang, D., Zhao, H., and Chen, Z. (2024c). Kvsharer: Efficient inference via layer-wise dissimilar kv cache sharing. *arXiv preprint arXiv:2410.18517*.

Yu, Z., Belinkov, Y., and Ananiadou, S. (2025). Back attention: Understanding and enhancing multi-hop reasoning in large language models. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V., editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 11257–11272, Suzhou, China. Association for Computational Linguistics.

Zeng, B., Song, S., Huang, S., Wang, Y., Li, H., He, Z., Wang, X., li, Z., and Lin, Z. (2026). PonderLM: Pretraining language models to ponder in continuous space. In *The Fourteenth International Conference on Learning Representations*.

Zhu, R.-J., Wang, Z., Hua, K., Zhang, T., Li, Z., Que, H., Wei, B., Wen, Z., Yin, F., Xing, H., Li, L., Shi, J., Ma, K., Li, S., Kergan, T., Smith, A., Qu, X., Hui, M., Wu, B., Min, Q., Huang, H., Zhou, X., Ye, W., Liu, J., Yang, J., Shi, Y., Lin, C., Zhao, E., Cai, T., Zhang, G., Huang, W., Bengio, Y., and Eshraghian, J. (2025). Scaling latent reasoning via looped language models. arXiv:2510.25741 [cs.LG].